

JSimple: uma ferramenta web didática para o ensino de conceitos de teoria de linguagens e compiladores

João Paulo Martyr Ragazzo, Luiz Eduardo da Silva

¹Bacharelado em Ciência da Computação – Universidade Federal de Alfenas (UNIFAL)
R. Gabriel Monteiro da Silva, 700 - Centro, Alfenas - MG, 37130-001 – Brasil

joao.ragazzo@sou.unifal-mg.edu.br, luiz.silva@unifal-mg.edu.br

Abstract. *The construction of compilers is fundamental to understanding the design of programming languages and is one of the most important courses in computer science. However, existing educational tools are limited to desktop interfaces and do not offer visualization of critical stages such as syntactic analysis. This work presents JSimple, an interactive web platform for teaching compilers, featuring visualization of syntax trees, step-by-step execution of the Simple language, and real-time feedback. The evaluation through expert inspection, based on Nielsen's heuristics and the LORI framework, resulted in an average of 4.0/5 for usability and 4.3/5 for pedagogical quality.*

Resumo. *A construção de compiladores é fundamental para compreender o design de linguagens de programação, sendo um dos cursos mais importantes da computação. Entretanto, ferramentas educacionais existentes são limitadas a interfaces desktop e não oferecem visualização de etapas críticas como a análise sintática. Este trabalho apresenta o JSimple, uma plataforma web interativa para ensino de compiladores, com visualização de árvores sintáticas, execução passo a passo da linguagem Simple e feedback em tempo real. A avaliação por inspeção especializada, baseada nas heurísticas de Nielsen e no framework LORI, resultou em média de 4,0/5 para usabilidade e 4,3/5 para qualidade pedagógica.*

1. Introdução

A digitalização alcança desde simples atividades cotidianas até processos industriais, dando origem à internet das coisas. Por trás de toda essa transformação digital, milhões de linhas de instruções bem definidas (algoritmos) permitem que máquinas executem ações, resolvam problemas e tomem decisões sem a interferência humana [Tan and Wang 2010].

A implementação de algoritmos é feita utilizando uma linguagem de programação: uma forma de escrita intermediária entre a linguagem natural e a linguagem de máquina. Entretanto, humanos escrevem e entendem apenas linguagens de programação. As máquinas, por outro lado, só conseguem interpretar instruções em formato binário (linguagem de máquina). Surge então a necessidade de traduzir a linguagem de programação em linguagem de máquina.

Essa transformação digital depende fundamentalmente de compiladores: programas especializados que traduzem linguagens de programação em código executável, permitindo que as máquinas executem as instruções escritas por programadores humanos [Alfred et al. 2007].

A construção de um compilador fornece a base fundamental para compreender o *design* de uma linguagem de programação, tornando o curso de compiladores um dos mais importantes dentro da ciência da computação [Stamenković and Jovanović 2023] [Baïna and Benatallah 2021]. Apesar da relevância da disciplina de teoria de linguagens e compiladores, a maioria dos estudantes aparenta não ser motivada ou interessada em estudar este tópico [Chesnevar et al. 2004]. Isso se deve não apenas à alta complexidade desse assunto, mas também à percepção intrínseca da matemática nesse ramo [Stamenković et al. 2020].

Estudos apontam que estudantes preferem métodos de aprendizado ativos em oposição a métodos de aprendizado passivo baseados apenas em literaturas [Stamenković et al. 2020] [Carstens et al. 2021]. A teoria de linguagens e compiladores envolve tópicos em um nível de abstração que torna difícil tanto aprender quanto ensinar [Chesnevar et al. 2004]. Como Mernik e Zumer [Mernik and Zumer 2003] apontam, os professores são constantemente desafiados a ensinar cada vez melhor. Dessa forma, recursos didáticos interativos que facilitem a complexidade de aprender e ensinar interessam a alunos e professores [Álvar Arnaiz-González et al. 2018].

Este trabalho apresenta o desenvolvimento e avaliação de uma plataforma web para auxiliar no processo de ensino-aprendizagem da disciplina de compiladores, alinhando-se com as preferências dos alunos por métodos de aprendizado ativo, como destacado por Stamenković, Jovanović e Chakraborty [Stamenković et al. 2020].

1.1. Objetivos gerais

Este trabalho tem como objetivo principal efetivar a construção e posterior estudo da eficácia de uma plataforma *web* para auxiliar no processo de ensino-aprendizagem da disciplina de compiladores.

1.2. Objetivos específicos

Para atingir o objetivo principal, este trabalho estabelece quatro objetivos específicos. Primeiramente, realizar uma revisão bibliográfica em busca de soluções semelhantes existentes na literatura. Em seguida, estudar ferramentas que possam auxiliar na implementação. O terceiro objetivo consiste em desenvolver e implementar a plataforma *web* para ensino de teoria de linguagem e compiladores. Por fim, avaliar a qualidade através de inspeção heurística de usabilidade (Nielsen) [Nielsen and Molich 1990] e LORI (Learning Object Review Instrument) [J. Vargo and Archambault 2003].

2. Revisão Bibliográfica

A disciplina de teoria de linguagens e compiladores apresenta desafios tanto para professores quanto alunos. Diversos pesquisadores e professores buscam estratégias e ferramentas para facilitar a compreensão desses conceitos. Nesta seção, apresentaremos estudos e soluções que têm se proposto a auxiliar o ensino desta disciplina, com foco em ferramentas que utilizam recursos interativos para apoiar a aprendizagem de linguagens formais e compiladores.

Lisa, uma ferramenta desktop desenvolvida por Mernik e Zumer [Mernik and Zumer 2003], também foi um *software* pioneiro destinado a auxiliar no processo de ensino-aprendizagem de teoria de linguagens e compiladores. Ao

contrário do JFLAP, essa ferramenta apresenta métodos para visualização das principais etapas do processo de compilação, como a análise léxica, sintática e semântica.

Por ser um aplicativo desktop, enfrenta problemas de compatibilidade para utilização em diferentes sistemas operacionais e requer instalação local. Como Stamenković, Jovanović e Chakraborty [Stamenković et al. 2020] destacam, *softwares* educacionais devem ser desenvolvidos em ambiente *web*, solucionando problemas de compatibilidade, instalação e acesso multiplataforma. Este trabalho adota essa abordagem, alinhando-se com essas observações, implementando também a funcionalidade de simulação passo a passo, semelhante a Lisa.

O Java Formal Languages and Automata Package (JFLAP) foi uma dos sistemas pioneiros, representando um marco no uso de *software* educacional para o ensino de linguagens formais e autômatos. Este *software* oferece um ambiente visual e interativo que facilita a compreensão de conceitos abstratos. Uma das inovações centrais do JFLAP é permitir que os alunos construam, simulem e transformem estruturas formais com *feedback* imediato, algo que os métodos tradicionais não permitem. Como destacado pelos autores, “os alunos podem verificar suas respostas e interagir com esses autômatos” [Rodger and Finley 2006].

Um estudo realizado com estudantes de ciência da computação mostrou que o uso do JFLAP contribuiu significativamente para o entendimento de conceitos teóricos, promovendo maior engajamento e melhor desempenho em avaliações [Silva et al. 2023]. A aplicação web desenvolvida neste trabalho segue a linha de pensamento do JFLAP ao oferecer interatividade, porém diferencia-se ao focar especificamente na linguagem Simples, execução passo a passo da MVS (Máquina Virtual Simples) e visualização da árvore sintática, funcionalidades não presentes no JFLAP.

Hiago Borges de Oliveira desenvolveu o Ambiente de Desenvolvimento Integrado Simples (ADIS) [Oliveira 2014], que permite estudar o processo de compilação e execução da linguagem Simples, incluindo a execução passo a passo do código. Diferentemente desse trabalho, o JSimples se destaca por portar essas funcionalidades para o ambiente web, além de oferecer a visualização da árvore sintática e da árvore de derivação da linguagem Simples.

Além disso, Arnaiz-González et al. [Álvar Arnaiz-González et al. 2018] desenvolveram o Seshat, uma plataforma web que complementa o JFLAP ao focar na análise léxica e na conversão de expressões regulares em autômatos finitos. Ao contrário do Seshat, o JSimples foca especialmente na demonstração da análise sintática e execução passo a passo da linguagem Simples.

Uma análise mais recente realizada por Stamenković, Jovanović e Chakraborty [Stamenković et al. 2020], que avaliou 16 ferramentas de simulação voltadas ao ensino de construção de compiladores, identificou que, embora ferramentas como o JFLAP e Lisa se destaquem pela interatividade, ambas carecem de funcionalidades de simulação para a etapa de análise sintática [Stamenković et al. 2020]. As ferramentas existentes, por sua vez, não satisfazem às necessidades de cursos introdutórios. Nesse contexto, este trabalho desenvolveu uma plataforma web focada na compreensão de conceitos sintáticos e execução passo a passo da linguagem Simples, com o objetivo de complementar as lacunas encontradas nas ferramentas já existentes.

Stamenković e Jovanović [Stamenković and Jovanović 2023], com base nas lacunas funcionais identificadas no estudo de Stamenković, Jovanović e Chakraborty [Stamenković et al. 2020], desenvolveram o ComVis: uma plataforma web destinada a simular os principais tópicos da disciplina de teoria de linguagens e compiladores. ComVis permite aos alunos explorar diferentes etapas e conceitos teóricos do processo de compilação, como autômatos finitos determinísticos, análise léxica, sintática e semântica. Embora o ComVis apresente maior abrangência de tópicos, o JSimples diferencia-se ao oferecer uma experiência aprofundada e centrada na linguagem Simples, permitindo que estudantes brasileiros explorem compilação com uma linguagem didática em português.

A análise das ferramentas educacionais revela três gaps principais que motivam o desenvolvimento do JSimples: (1) Gap de acessibilidade - ferramentas robustas como ADIS e Lisa são limitadas a ambientes desktop, exigindo instalação e restringindo acesso; (2) Gap de visualização sintática - nenhuma ferramenta web atual oferece visualização interativa de árvores sintáticas combinada com execução passo a passo; (3) Gap contextual - ausência de ferramentas web em português que integrem teoria e prática de compilação. A Tabela 1 sintetiza as principais características das ferramentas educacionais discutidas em comparação com o JSimples.

Ferramenta	Plataforma	Execução passo a passo	Linguagem específica	Visualização da árvore sintática
JFLAP	Desktop	×	×	×
Lisa	Desktop	✓	×	×
ADIS	Desktop	✓	✓	×
Seshat	Web	×	×	×
ComVis	Web	×	×	✓
JSimples	Web	✓	✓	✓

Tabela 1. Comparação entre ferramentas educacionais

Observa-se que o JSimples é a única ferramenta que combina plataforma web, execução passo a passo, foco em linguagem específica didática e visualização de árvore sintática, preenchendo lacunas identificadas nas soluções existentes.

Por fim, a revisão bibliográfica evidencia uma evolução de ferramentas *desktop* especializadas (JFLAP, Lisa, ADIS) para soluções web multiplataforma (ComVis, Seshat), porém com persistência de lacunas funcionais específicas. O JSimples posiciona-se como síntese dessa evolução, combinando acessibilidade web, profundidade pedagógica centrada na linguagem Simples, árvore sintática e de derivação e contextualização para o ensino brasileiro.

3. Ambiente de Desenvolvimento Integrado Simples (ADIS)

3.1. Linguagem Simples

A linguagem Simples [Oliveira 2014] é uma linguagem de programação didática desenvolvida por Hiago Borges de Oliveira, criada para facilitar o ensino de conceitos fundamentais de programação, compiladores e arquitetura de computadores. O processo

de execução da linguagem segue um fluxo bem definido: o código-fonte escrito em linguagem Simples é traduzido pelo Compilador Simples para instruções MVS (Máquina Virtual Simples), que são interpretadas e executadas pela máquina virtual, gerando o resultado final.

Esta linguagem foi projetada com comandos em português, tornando-a mais intuitiva para estudantes brasileiros, e inclui funcionalidades essenciais como declaração de variáveis (tipos inteiro e lógico), comandos de entrada e saída (leia e escreva), estruturas condicionais (se-então-senão), laços de repetição (enquanto-faça), funções e procedimentos (com passagem de parâmetros de valores ou referência a variáveis), e suporte a expressões aritméticas, relacionais e lógicas. Esta abordagem simplificada permite que os alunos compreendam de forma prática a relação entre uma linguagem de alto nível, o processo de compilação e a execução em nível de máquina.

3.2. Definição da Máquina Virtual Simples (MVS)

A MVS é um projeto de máquina virtual semelhante a Máquina para Execução Pascal [Kowaltowski 1983], sendo uma máquina de pilha, característica que proporciona um ambiente ideal para programas com recursividade e estruturas aninhadas.

Essa arquitetura de pilha reflete-se na organização da memória, que é dividida em duas regiões: o Vetor P armazena as instruções do programa, e a Pilha M armazena os valores manipuláveis durante a execução. Essas regiões trabalham de forma integrada, onde as instruções dispostas no Vetor P operam sobre os valores armazenados na Pilha M.

Para controlar a execução e o acesso a essas regiões de memória, a MVS possui três registradores essenciais: o registrador *I* aponta para a próxima instrução a ser executada no Vetor P; o registrador *S* indica o topo da Pilha M; e o registrador *D* aponta para a base do escopo, sendo útil em gerenciamento de escopos em chamadas de funções e procedimentos.

No que se refere ao conjunto de instruções, a MVS é composta por 31 palavras-chave mnemônicas, projetadas para tornar a operação realizada por cada instrução explícita, facilitando a compreensão, depuração de programas compilados para essa arquitetura e o entendimento de cada instrução MVS. Todas as 31 instruções são explicadas na Tabela 2.

Tabela 2: Instruções da Máquina Virtual Simples (MVS)

Instrução	Descrição
INPP	Inicia a execução programa principal.
FIMP	Finaliza a execução do programa.
AMEM <i>k</i>	Aloca <i>k</i> espaços na memória.
CRCT <i>k</i>	Carrega o valor constante <i>k</i> no topo da Pilha M.
CRVG <i>n</i>	Carrega o valor da variável global armazenada no endereço <i>n</i> no topo da Pilha M.
ARZG <i>n</i>	Armazena o valor do topo da Pilha M no endereço global <i>n</i> .
DSVS <i>p</i>	Desvia incondicionalmente o ponteiro <i>I</i> para a posição <i>p</i> do Vetor P.

Instrução	Descrição
DSVF p	Caso o valor no topo da Pilha M seja falso, desvia o ponteiro I para a posição p do Vetor P.
LEIA	Lê um valor no console e o armazena no topo da Pilha M.
ESCR	Escreve no console o valor contido no topo da Pilha M.
CMME	Compara se o valor no topo da Pilha M -1 é menor que o valor no topo da Pilha M e empilha o resultado.
CMMA	Compara se o valor no topo da Pilha M -1 é maior que o valor no topo da Pilha M e empilha o resultado.
CMIG	Compara se o valor no topo da Pilha M -1 é igual ao valor no topo da Pilha M e empilha o resultado.
DISJ	Realiza operação de disjunção entre os dois elementos no topo da Pilha M e empilha o resultado.
CONJ	Realiza operação de conjunção entre os dois elementos no topo da Pilha M e empilha o resultado.
NEGA	Realiza a operação de negação no valor do topo da Pilha M e empilha o resultado.
SOMA	Soma os dois elementos no topo da Pilha M e empilha o resultado.
SUBT	Subtrai o topo da Pilha M -1 pelo valor no topo da Pilha M e empilha o resultado.
MULT	Multiplica os dois elementos no topo da Pilha M e empilha o resultado.
DIVI	Divide o topo da Pilha M -1 pelo valor no topo da Pilha M e empilha o resultado.
NADA	Não realiza nenhuma operação. Útil para instruções de desvio.
CRVL n	Carrega o valor da variável local da posição n da Pilha M.
ARZL n	Armazena o valor do topo da Pilha M na posição n da Pilha M.
CREL n	Carrega o endereço local n no topo da Pilha M.
CREG n	Carrega o endereço global n no topo da Pilha M.
CRVI n	Carrega o valor armazenado no endereço contido na posição n da Pilha M.
ARMI n	Armazena o valor do topo da Pilha M no endereço contido na posição n da Pilha M.
SVCP	Empilha o contador de programa no topo da Pilha M.
ENSP	Indica a entrada de um subprograma.
RTSP n	Retorno do subprograma para o endereço n no Vetor P.
DMEM n	Desaloca n posições da memória.

3.3. Portabilidade do ADIS ao ambiente web

A ferramenta didática desenvolvida em Java por Hiago Borges para o ensino de lógica de programação, compiladores e arquitetura de computadores, utilizando linguagem Simples, é limitada apenas a ambientes *desktop*, não havendo portabilidade para outros dispositivos [Oliveira 2014]. A necessidade da instalação da Java Virtual Machine (JVM) também se torna um problema, especialmente em dispositivos com recursos limitados ou políticas restritivas de instalação de software. Essas limitações motivam o desenvolvimento de uma solução web mais acessível e multiplataforma.

Nesse contexto, o JSimples foi portado para o ambiente web, o que elimina a

necessidade de instalação local e possibilita a integração com plataformas de ensino por meio do padrão SCORM (*Sharable Content Object Reference Model*)¹. Isso torna o acesso a ferramenta mais acessível e multiplataforma, alinhando-se com as observações feitas por Stamenković, Jovanović e Chakraborty [Stamenković et al. 2020].

4. Metodologia de Desenvolvimento

4.1. Tecnologias e Arquitetura

O desenvolvimento do JSimples baseou-se em uma abordagem *client-side*, ou seja, sem a necessidade de um *back-end* tradicional. Foram utilizadas tecnologias *web* atuais que garantem a execução direta no navegador sem dependências externas, garantindo uma ferramenta acessível e multiplataforma. A escolha do JavaScript como linguagem principal fundamentou-se em sua ampla adoção no desenvolvimento web e compatibilidade com navegadores modernos. HTML5 e CSS3 complementam a base tecnológica, fornecendo a base semântica e visual da aplicação.

4.2. Frameworks e Bibliotecas

Para otimizar o desenvolvimento e manutenção do código, adotou-se o React [Meta Platforms, Inc. 2025] como *framework* principal. Esta escolha fundamentou-se na capacidade de componentização modular da interface, permitindo maior reutilização de componentes e encapsulamento da lógica de negócio. O React também oferece um ciclo de vida bem definido para o gerenciamento de estado, facilitando o controle das interações necessárias em um ambiente de execução passo a passo interativo.

Para componentes de interface, utilizamos a biblioteca *Ant Design* [Ant UED 2015], que fornece componentes React prontos e responsivos. Esta escolha garante padronização visual através de um *design system* consistente e acelera o desenvolvimento ao disponibilizar componentes básicos de interface já implementados. A responsividade nativa dos componentes também assegura uma experiência adequada em diferentes tamanhos de tela.

A biblioteca *Styled Components* [Maddern and Stoiber 2016] implementou a estilização do JSimples, que proporciona uma abordagem CSS-in-JS com escopo isolado. Esta solução elimina conflitos de nomenclatura CSS comuns de aplicações, além de melhorar a organização de estilos, aspectos fundamentais para a manutenibilidade do projeto.

4.2.1. Gerenciamento de Estados

Para o gerenciamento centralizado de estado da aplicação, foi adotada a Context API nativa do React. Esta solução permite o compartilhamento de dados entre os módulos principais: Compilador Simples, Máquina Virtual Simples e Interface.

O Context API gerencia diversos estados: código-fonte, código MVS, árvore sintática e de derivação, sinais de execução, entradas do usuário e retornos da execução.

¹Padrão internacional que permite integrar conteúdos educacionais em plataformas de ensino como Moodle e Canvas, facilitando a distribuição e o acesso centralizado de aplicativos educacionais.

Esta arquitetura garante sincronização consistente, propagando alterações automaticamente entre componentes e refletindo interações em tempo real. A Figura 1 ilustra a organização da arquitetura do JSimples, evidenciando o fluxo de dados entre os módulos e o papel central do Context API na integração dos componentes.

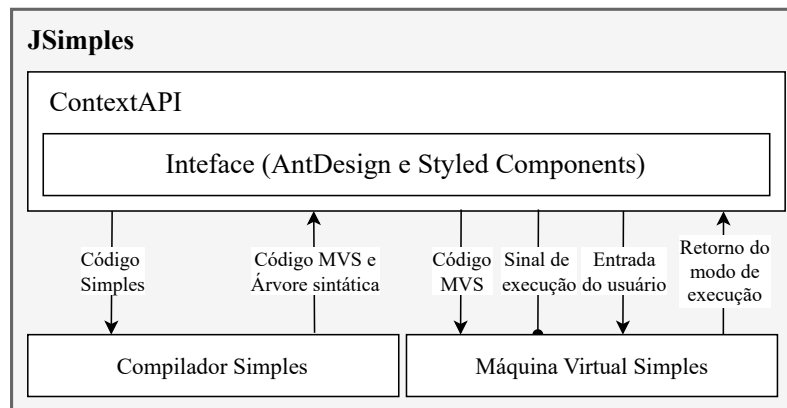


Figura 1. Arquitetura do JSimples

4.2.2. Implementação do editor de código

Para o editor de código, foi integrada a biblioteca CodeMirror [Haverbeke 2018], utilizando um *wrapper* para o React. A fim de adaptar o editor às especificidades da linguagem Simples, desenvolveu-se um *plugin* adaptado a sintaxe da linguagem Simples, que implementa *syntax highlighting* específico para a sintaxe da linguagem, como demonstrado na Figura 2. Esta personalização melhora a experiência do usuário ao proporcionar *feedback* visual imediato sobre palavras-chave utilizadas no programa, facilitando a identificação da estrutura, erros de sintaxe e a compreensão geral do programa.

```

1 programa teste
2     inteiro a b
3     logico c d
4 inicio
5     a <- 1
6     a <- a * 3
7     escreva a
8 fimprograma

```

Figura 2. Implementação do *syntax highlighting* no editor CodeMirror

4.2.3. Implementação da representação da árvore sintática e de derivação

Para a implementação da árvore sintática e da árvore de derivação, foi utilizada a biblioteca React D3 Tree [Kremer 2017], que permite a construção de árvores hierárquicas interativas através da integração entre React e D3.js. Esta biblioteca oferece renderização eficiente de estruturas arbóreas complexas com suporte nativo a zoom, pan e navegação interativa, características essenciais para visualizar árvores sintáticas e árvores de derivação com múltiplos níveis.

A representação visual da árvore sintática e da árvore de derivação gerada pelo compilador possibilita ao estudante compreender como o código-fonte em linguagem Simples é estruturado hierarquicamente durante o processo de análise sintática. Cada nó da árvore corresponde a uma construção sintática da linguagem, como expressões, comandos de controle ou declarações de variáveis, conforme ilustrado na Figura 3. A interatividade proporcionada pela biblioteca permite expandir e colapsar subárvores, facilitando a análise de trechos específicos do programa sem perder o contexto da estrutura global.

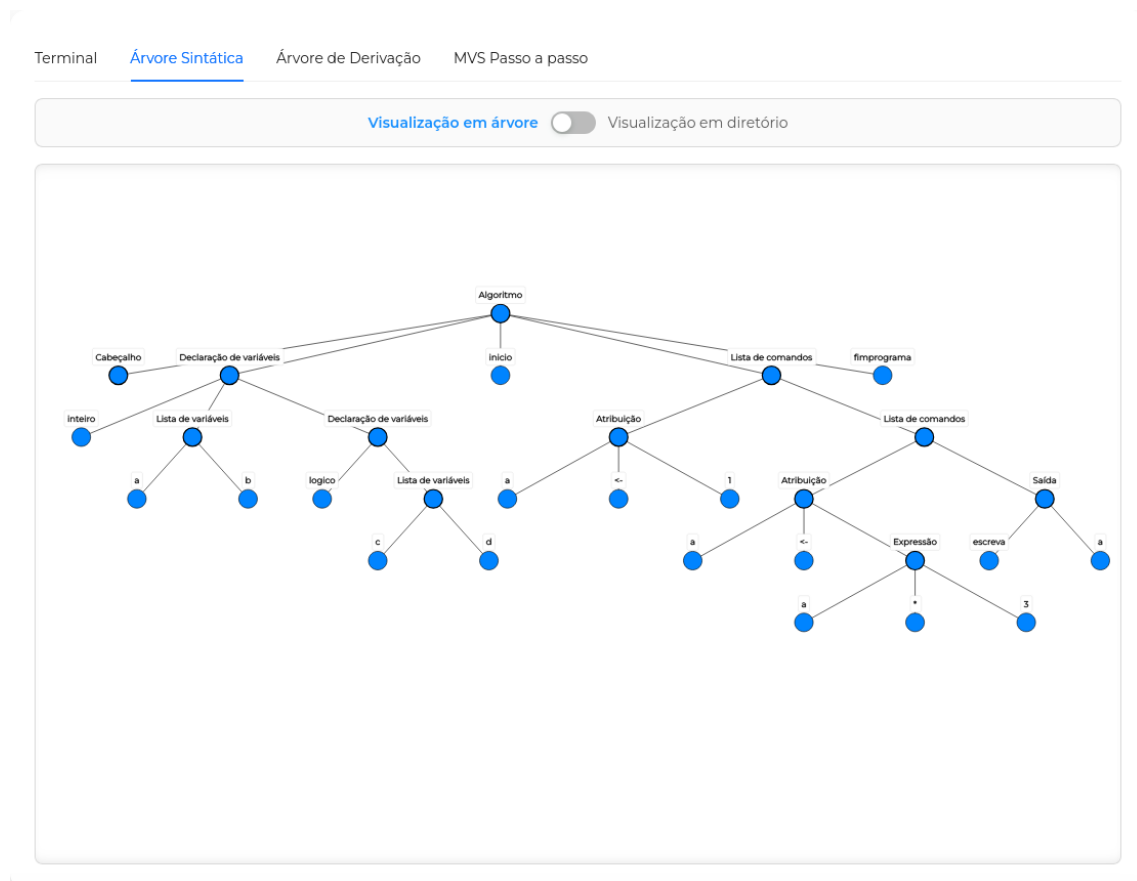


Figura 3. Árvore sintática da gramática Simples gerada pelo JSimples ²

4.2.4. Implementação da Tabela de Símbolos

A tabela de símbolos é uma estrutura que visualiza como as variáveis são identificadas pelo compilador em tempo de compilação, exibindo informações como tipo, identificador, endereço, escopo, rótulo, categoria e parâmetros (para funções e procedimentos) conforme ilustrado pela Figura 4. A implementação foi desenvolvida utilizando componentes da biblioteca Ant Design, com adaptações para aumentar a responsividade em dispositivos móveis.

²Árvore sintática gerada pela compilação do programa Simples disponível na Figura 2.

	Tipo	Identificador	Endereço	Escopo	Rótulo	Categoria	Parâmetros
	Inteiro	a	0	• GLOBAL	-	• Variável	-
	Inteiro	b	1	• GLOBAL	-	• Variável	-
	Lógico	c	2	• GLOBAL	-	• Variável	-
	Lógico	d	3	• GLOBAL	-	• Variável	-

Figura 4. Tabela de símbolos gerada pela compilação de um código Simples ³

4.3. Implementação do Compilador

O compilador do JSimples foi desenvolvido utilizando Jison [Zach Carter 2009], um gerador de analisadores sintáticos inspirado no Bison [Free Software Foundation 2025] e Yacc [Johnson 1975]. A escolha desta ferramenta baseou-se em sua integração nativa com aplicações web, possibilitando a execução direta em ambiente de navegador sem dependências externas.

Para sua utilização, criamos uma especificação de gramática em formato `.jison`, a partir da qual o Jison gera automaticamente um analisador sintático em JavaScript. Este analisador é então integrado ao JSimples, onde processa a entrada do usuário e produz o código MVS através da construção da árvore sintática, como demonstrado na Figura 5.

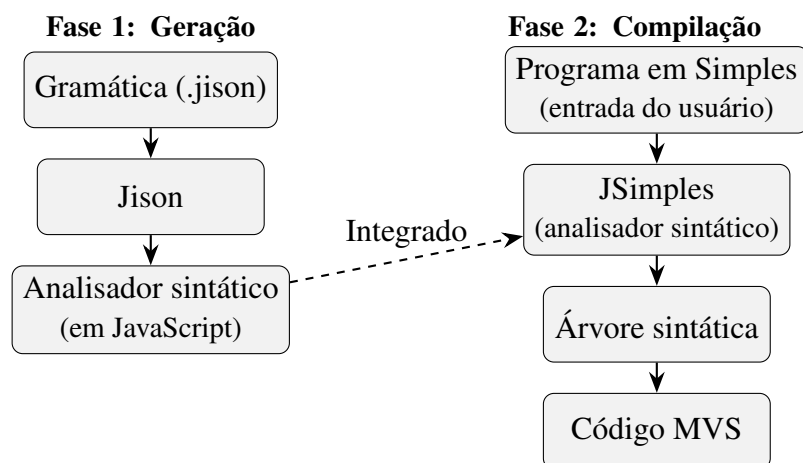


Figura 5. Processo de geração e uso do analisador sintático.

Vale ressaltar que o Jison apresenta uma limitação em relação ao Bison/Yacc tradicional: a ausência de suporte para *mid-rule actions* ⁴. Esta restrição exigiu a reestruturação da gramática original da linguagem Simples, utilizando regras auxiliares menores para garantir que ações semânticas fossem executadas apenas ao final de cada

³Tabela de símbolos gerada pela compilação do programa Simples disponível na Figura 2.

⁴*Mid-rule actions* são blocos de código executados durante o reconhecimento de uma regra de produção, permitindo processamento semântico incremental antes que toda a regra seja derivada.

regra auxiliar. Apesar do aumento no número de regras da gramática, esta abordagem manteve o resultado final esperado.

4.4. Implementação da Máquina Virtual Simples

Implementamos a MVS em JavaScript seguindo a especificação original [Oliveira 2014], mantendo fidelidade à arquitetura de pilha e ao conjunto de 31 instruções.

O ciclo de execução segue o modelo *fetch-decode-execute* tradicional: busca da instrução no Vetor P, identificação do tipo de operação e execução sobre a Pilha M, repetindo-se até encontrar a instrução FIMP. A Figura 6 ilustra o funcionamento da MVS.

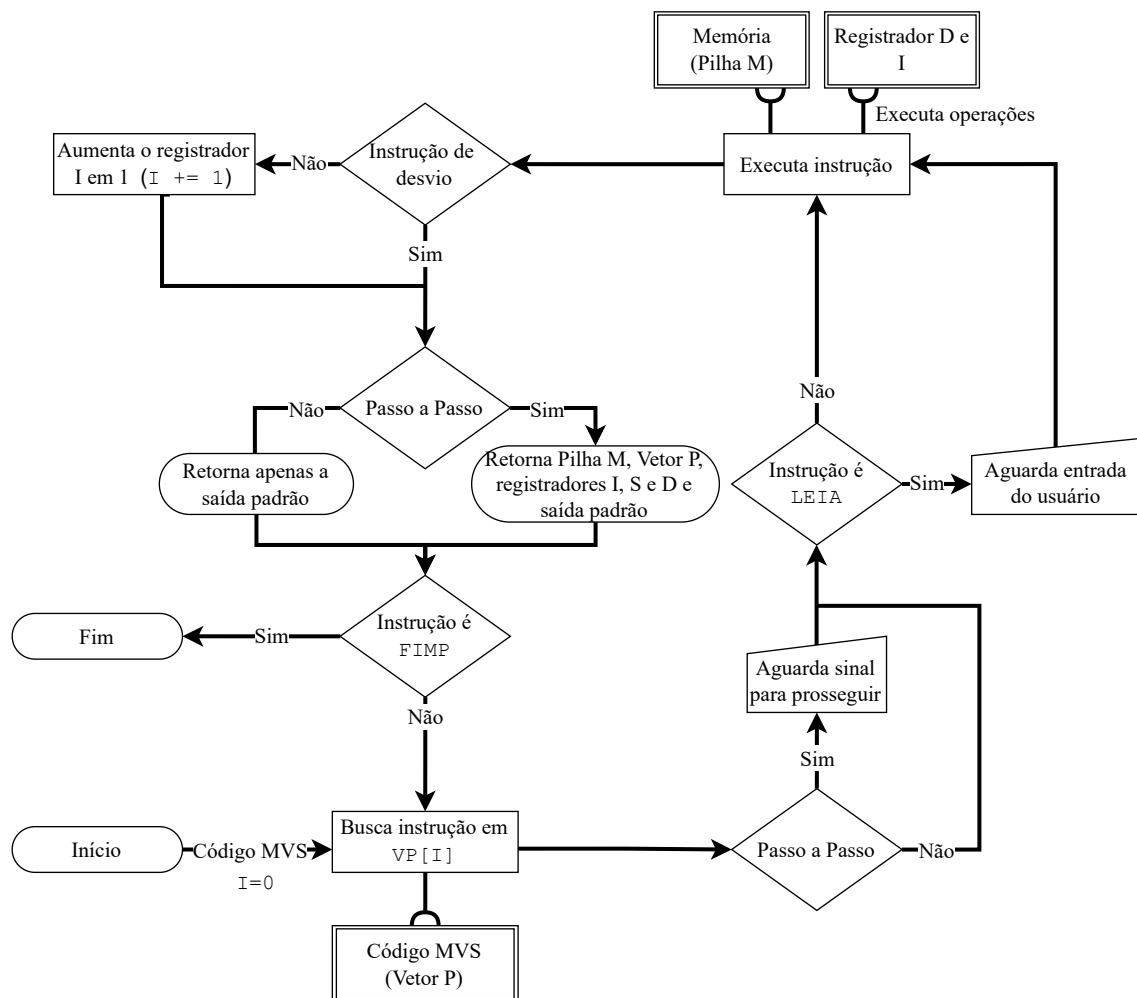


Figura 6. Funcionamento da Máquina Virtual Simples (MVS)

4.4.1. Execução Assíncrona e Controle de Fluxo

A execução é gerenciada de forma assíncrona e não-bloqueante, permitindo que a interface permaneça responsiva mesmo durante a execução de iterações extensas. O controle de fluxo permite pausar, retomar ou interromper a execução conforme necessário.

Para operações de entrada, foi implementado um sistema assíncrono que suspende temporariamente a execução, aguarda a entrada do usuário via interface gráfica, e retoma

o processamento após a captura do valor. Esta abordagem garante que a aplicação não bloqueie enquanto aguarda a interação do usuário.

4.4.2. Modo de Execução Passo a Passo

A MVS suporta dois modos de execução: contínua e passo a passo. O modo passo a passo permite que o usuário avance a execução uma instrução por vez. Após cada instrução, a execução é pausada até que o usuário decida prosseguir. A cada avanço, o estado completo da máquina (registradores I, S e D, conteúdo da pilha, tabela de instrução MVS e o indicador da próxima instrução) é exibido na interface, permitindo o usuário acompanhar o estado da máquina em tempo real. O modo passo a passo é mostrado na Figura 7.

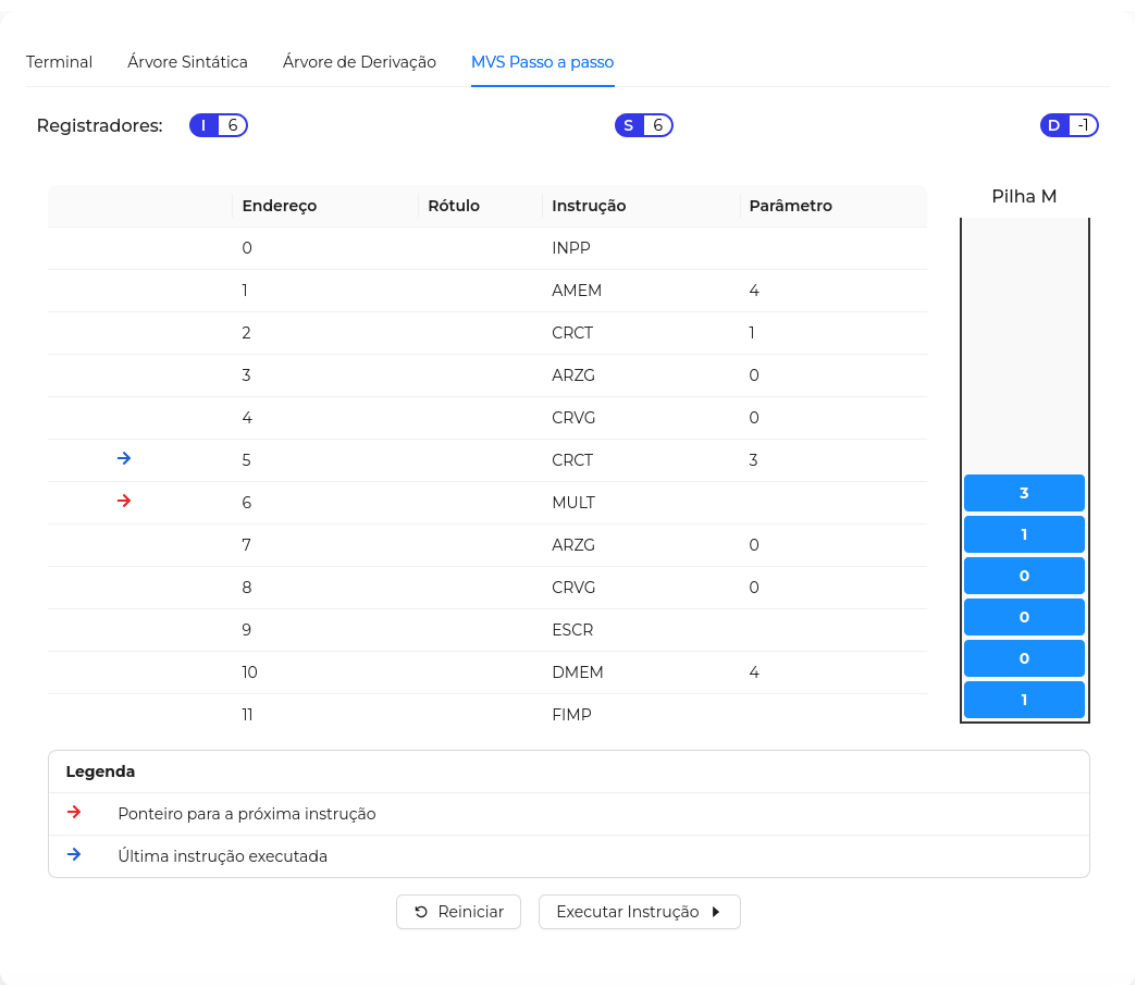


Figura 7. Interface do modo passo a passo do JSimples ⁵

4.5. Testes e Validação

A validação da aplicação seguiu uma metodologia contemplando testes de compatibilidade em múltiplos navegadores em suas versões mais recentes. A responsividade da

⁵Execução passo a passo gerada pela compilação do programa Simples disponível na Figura 2

aplicação também foi validada em resoluções variadas, desde telas de 320px de largura até monitores desktop com resolução de 1920px. Esta abordagem assegurou que a interface permanecesse funcional e esteticamente adequada em diferentes contextos de uso, garantindo a portabilidade e acesso multiplataforma do aplicativo.

5. Avaliação da plataforma

A avaliação da plataforma JSimples combinou *frameworks* complementares de avaliação. Esta estratégia permite analisar tanto aspectos técnicos de usabilidade quanto dimensões pedagógicas específicas de *software* educacional.

5.1. Metodologia de Avaliação

A avaliação de usabilidade é fundamentada nas 10 heurísticas de Nielsen [Nielsen and Molich 1990], focando em princípios gerais de design de interface e interação humano-computador. Por outro lado, a avaliação pedagógica é baseada no instrumento LORI (*Learning Object Review Instrument*) [J. Vargo and Archambault 2003], específico para objetos de aprendizagem.

Esta abordagem é justificada pela natureza híbrida do JSimples: trata-se simultaneamente de uma aplicação web que deve ser usável (domínio de interface humano-computador) e de uma ferramenta educacional que deve promover aprendizagem efetiva (domínio pedagógico). A análise isolada de apenas uma dimensão seria insuficiente para avaliar adequadamente uma ferramenta didática interativa.

5.1.1. Protocolo de Avaliação

A avaliação foi conduzida pelo autor deste trabalho, adotando perspectiva de especialista com conhecimento em desenvolvimento de software e teoria de linguagens e compiladores. Para cada *framework*, foram seguidos os seguintes procedimentos:

Heurísticas de Nielsen: Navegação sistemática pela interface exercitando todas as funcionalidades principais (compilação, execução contínua, execução passo a passo, visualização de árvore sintática e da árvore de derivação).

LORI: Análise individual de cada dimensão pedagógica, atribuindo pontuação fundamentada em evidências observadas na ferramenta, seguindo escala Likert de 5 pontos conforme recomendado pelos autores.

5.2. Avaliação de Usabilidade (Heurísticas de Nielsen)

As heurísticas de usabilidade de Nielsen constituem dez princípios gerais para design de interface que permitem identificar problemas de usabilidade através de inspeção sistemática. As heurísticas avaliadas são:

1. **Visibilidade do status do sistema:** O sistema deve sempre manter os usuários informados sobre o que está acontecendo, através de *feedback* em tempo real.
2. **Correspondência entre sistema e mundo real:** O sistema deve falar a linguagem dos usuários, com palavras, frases e conceitos familiares, ao invés de termos técnicos.

3. **Controle e liberdade do usuário:** Usuários frequentemente escolhem funções por engano e precisam de uma "saída de emergência" claramente marcada.
4. **Consistência e padrões:** Usuários não devem ter que se perguntar se palavras, situações ou ações diferentes significam a mesma coisa.
5. **Prevenção de erros:** Ainda melhor que boas mensagens de erro é um design cuidadoso que previne problemas.
6. **Reconhecimento em vez de memorização:** Minimizar a carga de memória do usuário tornando objetos, ações e opções visíveis.
7. **Flexibilidade e eficiência de uso:** Aceleradores invisíveis para usuários novatos podem aumentar a velocidade de interação para especialistas.
8. **Design estético e minimalista:** Diálogos não devem conter informação irrelevante ou raramente necessária.
9. **Ajudar usuários a reconhecer, diagnosticar e recuperar erros:** Mensagens de erro devem ser expressas em linguagem simples, indicar o problema precisamente e sugerir soluções.
10. **Ajuda e documentação:** Embora seja melhor que o sistema possa ser usado sem documentação, pode ser necessário fornecer ajuda e documentação.

A aplicação de cada heurística ao JSimples é detalhada a seguir.

5.2.1. Pontos Fortes Identificados

A avaliação revelou conformidade plena em aspectos fundamentais de usabilidade, conforme detalhado a seguir:

H1 - Visibilidade do Status do Sistema (5/5): A plataforma fornece *feedback* imediato durante o processo de compilação e execução. Durante compilação, erros são exibidos instantaneamente. No modo passo a passo (Figura 6), a instrução MVS atual é destacada visualmente e o estado completo da máquina (registradores I, S, D e conteúdo da Pilha M) é atualizado de forma síncrona, permitindo ao estudante acompanhar cada transição de estado.

H2 - Correspondência entre Sistema e Mundo Real (5/5): A utilização de comandos em português na linguagem Simples reduz significativamente a barreira linguística comparado a ferramentas em inglês como JFLAP e ComVis. Esta escolha de design permite que estudantes brasileiros concentrem esforços cognitivos nos conceitos de compilação ao invés de tradução mental de termos estrangeiros. A interface também emprega terminologia educacional familiar, alinhando-se com vocabulário usado em sala de aula.

H4 - Consistência e Padrões (5/5): A adoção da biblioteca Ant Design garante padrões visuais e comportamentais consistentes em toda a interface. Botões com funções similares mantêm posicionamento, cor e estilo uniformes. A organização dos seis painéis principais (editor de código, console, árvore sintática, árvore de derivação, execução passo a passo e tabela de símbolos) permanece constante durante toda a interação, facilitando a navegação e reduzindo carga cognitiva.

H8 - Design Estético e Minimalista (5/5): A interface mantém foco em elementos essenciais para o fluxo de trabalho: escrita de código, visualização de estrutura

sintática e acompanhamento de execução. Não há elementos decorativos supérfluos, propagandas ou funcionalidades secundárias que distraiam o estudante da tarefa principal. O layout de seis painéis otimiza uso do espaço de tela sem sobrecarregar visualmente o usuário.

H9 - Ajudar a Recuperar de Erros (5/5): As mensagens de erro de compilação são específicas e informativas, indicando linha e coluna do problema, tipo do erro (léxico, sintático, semântico) e descrição clara em português. Por exemplo, ao detectar declaração duplicada de variável, o sistema informa a posição e o símbolo já declarado, permitindo localização e correção rápida. Esta clareza é particularmente importante para estudantes iniciantes, que se beneficiam de *feedback* pedagógico ao invés de mensagens técnicas.

5.2.2. Oportunidades de Melhoria

Cinco heurísticas apresentaram conformidade parcial, indicando aspectos que, embora não comprometam a funcionalidade essencial, poderiam ser aprimorados:

H3 - Controle e Liberdade do Usuário (3/5): A plataforma permite pausar, retomar e interromper a execução através de botões claramente identificados. Entretanto, a ausência de funcionalidade para "voltar passo" limita o controle total do usuário sobre a navegação durante execução incremental. Atualmente, para reanalisar um trecho específico já executado, o estudante deve reiniciar a execução completa desde o início. Implementar histórico navegável de estados da MVS, permitindo retroceder para qualquer ponto anterior da execução. Esta funcionalidade facilitaria análise comparativa de estados e depuração de lógica.

H5 - Prevenção de Erros (3/5): A plataforma implementa múltiplos mecanismos preventivos, como *syntax highlighting* (Figura 2), que destaca palavras-chave da linguagem Simples em cores distintas, permitindo identificação visual imediata de comandos válidos. Estas características reduzem erros básicos de sintaxe e uso inadequado da interface. Entretanto, o editor não implementa análise estática em tempo real (*linting*): erros sintáticos só são detectados após tentativa de compilação, ao invés de serem sinalizados visualmente durante digitação. Esta limitação reduz a prevenção proativa de erros básicos.

H6 - Reconhecimento em vez de Memorização (3/5): A árvore sintática e de derivação apresentam rótulos claros para cada nó (comandos, expressões, declarações), e o *syntax highlighting* facilita reconhecimento visual de estruturas no código-fonte. Porém, o painel de execução MVS exibe apenas mnemônicos das instruções sem explicação contextual. Estudantes em primeiro contato com arquitetura de pilha podem não reconhecer imediatamente o significado de cada instrução. Para melhorar a compreensão de novos estudantes, dicas explicativas que exibam breve descrição sobre a implicação de uma instrução MVS sobre o estado da máquina.

H7 - Flexibilidade e Eficiência de Uso (4/5): O JSimples oferece dois modos de execução adequados a diferentes necessidades: execução contínua para visualização rápida de resultado final, e execução passo a passo para análise detalhada de cada instrução. Esta dualidade atende tanto usuários novatos quanto experientes. Entretanto, a ausência de atalhos de teclado força uso exclusivo do mouse, reduzindo eficiência para usuários avançados.

H10 - Ajuda e Documentação (2/5): A plataforma não oferece documentação sobre o código MVS e a sintaxe da linguagem Simples. Não há tutorial interativo de primeiro uso, ajuda contextual inline, ou exemplos de código integrados à interface. Estudantes sem familiaridade prévia podem enfrentar curva de aprendizado inicial em relação ao uso da linguagem Simples.

5.2.3. Síntese da Avaliação de Usabilidade

A análise heurística resultou em média de 4/5 para usabilidade. Os pontos fortes concentram-se em aspectos fundamentais: feedback claro e constante (H1), linguagem adequada ao público brasileiro (H2) e mensagens de erro pedagógicas (H9). A consistência visual (H4) e o minimalismo (H8) contribuem para interface limpa que não sobrecarrega estudantes durante processo de aprendizagem.

As oportunidades de melhoria identificadas (H3, H5, H6, H7, H10) referem-se predominantemente a funcionalidades avançadas que, embora desejáveis para otimizar experiência de usuários experientes, não impedem o uso efetivo da ferramenta por seu público-alvo principal: estudantes de graduação em disciplina introdutória de compiladores. A ausência de navegação reversa (H3) e atalhos de teclado (H7) representa limitação para cenários avançados de depuração, mas não compromete o fluxo básico de aprendizagem. Similarmente, *tooltips* explicativos (H6) e tutorial interativo (H10) aprimorariam curva de aprendizado inicial, mas não são impeditivos.

Heurística	Nota	Observação
H1: Visibilidade do <i>status</i> do sistema	5	<i>Feedback</i> em tempo real, atualização síncrona
H2: Correspondência mundo real	5	Linguagem Simples com comandos em português
H3: Controle e liberdade do usuário	3	Ausência de navegação reversa (voltar passo)
H4: Consistência e padrões	5	Biblioteca <i>Ant Design</i> garante uniformidade
H5: Prevenção de erros	3	Ausência de <i>linting</i> no editor de código
H6: Reconhecimento vs memorização	3	Faltam <i>tooltips</i> sobre instruções MVS
H7: Flexibilidade e eficiência de uso	4	Ausência de atalhos de teclado
H8: Design estético e minimalista	5	Interface limpa focada em essenciais
H9: Recuperação de erros	5	Mensagens específicas com linha e coluna
H10: Ajuda e documentação	2	Documentação básica, pode expandir
Média Geral	4	

Tabela 3. Avaliação heurística do JSimples segundo Nielsen

5.3. Avaliação Pedagógica (LORI)

O instrumento LORI (*Learning Object Review Instrument*) é um *framework* estabelecido para avaliação de objetos de aprendizagem. Este instrumento avalia nove dimensões pe-

pedagógicas em escala Likert de 1 a 5 pontos, onde 1 representa qualidade inadequada e 5 representa qualidade excelente. As dimensões avaliadas são:

1. **Qualidade do Conteúdo:** Veracidade, precisão, apresentação equilibrada de ideias e nível apropriado de detalhe.
2. **Alinhamento com Objetivos de Aprendizagem:** Correspondência entre objetivos, atividades, avaliações e características dos aprendizes.
3. **Feedback e Adaptação:** Capacidade de adaptar-se às respostas do aprendiz e fornecer *feedback* significativo.
4. **Motivação:** Capacidade de motivar e interessar a população-alvo identificada.
5. **Design de Apresentação:** Qualidade do design da informação visual e auditiva para facilitar processamento cognitivo.
6. **Usabilidade da Interação:** Facilidade de navegação, previsibilidade da interface e qualidade dos recursos de ajuda.
7. **Acessibilidade:** Design de controles e formato de apresentação para acomodar aprendizes com deficiências.
8. **Reusabilidade:** Capacidade de uso em diferentes contextos de aprendizagem e com diferentes alunos.
9. **Conformidade com Padrões:** Adesão a padrões e especificações internacionais.

A aplicação de cada dimensão ao JSimples é detalhada a seguir.

5.4. Pontos Fortes Identificados

A avaliação revelou excelência em aspectos fundamentais pedagógicos, conforme detalhado a seguir:

D1 - Qualidade do Conteúdo (5/5): A plataforma apresenta implementação fiel à especificação original da linguagem Simples e da MVS. O conteúdo apresentado nas visualizações corresponde ao comportamento esperado conforme documentação original. A terminologia utilizada é consistente com a literatura de compiladores. Não há simplificações excessivas que comprometam a correção técnica, nem detalhes que sobrecarreguem cognitivamente o aprendiz.

D2 - Alinhamento com Objetivos de Aprendizagem (5/5): A ferramenta foi projetada especificamente para atender aos objetivos de aprendizagem da disciplina de teoria de linguagens e compiladores. Cada funcionalidade mapeia-se diretamente a um objetivo pedagógico específico: a visualização da árvore sintática e árvore de derivação permite compreender análise sintática descendente; a execução passo a passo possibilita entender arquitetura de pilha e ciclo *fetch-decode-execute*; o código MVS gerado demonstra tradução de alto nível para instruções de máquina.

D4 - Motivação (5/5): A visualização interativa da árvore sintática e da árvore de derivação transforma conceito abstrato em representação visual. O modo de execução passo a passo permite que estudantes vejam o resultado de suas ações, proporcionando sensação de controle e descoberta ativa. A possibilidade de escrever, compilar e executar programas próprios em linguagem com comandos em português reduz barreiras linguísticas e facilita experimentação. O *feedback* visual imediato mantém engajamento durante exploração. Esta abordagem alinha-se com preferências estudantis por métodos ativos de aprendizagem identificadas por Stamenković et al [Stamenković et al. 2020].

D5 - Design de Apresentação (5/5): A interface utiliza hierarquia visual para organizar informações. O layout de seis painéis principais (editor de código, console, árvore sintática, árvore de derivação, execução passo a passo da MVS e tabela de símbolos) distribui informações relacionadas sem competição visual. O *syntax highlighting* diferencia elementos sintáticos através de cores consistentes, facilitando reconhecimento de padrões. A árvore sintática e árvore de derivação utilizam representações hierárquicas espaciais que refletem a estrutura de derivação gramatical. Durante execução passo a passo, a instrução atual é destacada, direcionando atenção do estudante. Os registradores e conteúdo da pilha são apresentados de forma organizada, reduzindo carga cognitiva necessária para interpretação do estado da máquina.

D8 - Reusabilidade (5/5): O JSimple demonstra reusabilidade em múltiplos contextos educacionais. A plataforma pode ser utilizada tanto em disciplinas introdutórias de lógica de programação (focando apenas na escrita e execução de programas Simple) quanto em cursos de compiladores (explorando análise sintática e geração de código). A arquitetura web multiplataforma reduz restrições de sistema operacional ou hardware específico. A conformidade com padrão SCORM possibilita integração com sistemas de gerenciamento de aprendizagem, facilitando adoção institucional.

5.5. Oportunidades de Melhoria

Quatro dimensões apresentaram desempenho satisfatório, porém revelaram margem para evolução em aspectos específicos:

D3 - Feedback e Adaptação (4/5): A plataforma oferece feedback imediato e específico durante compilação, indicando tipo de erro (léxico, sintático, semântico), linha, coluna e descrição em português. Durante execução passo a passo, o estudante visualiza mudanças de estado em tempo real. Entretanto, o sistema não conta com um sistema de *linting* para o editor de código, o que poderia facilitar o entendimento para novos estudantes.

D6 - Usabilidade da Interação (4/5): A navegação entre modos de execução (contínua vs. passo a passo) é clara e intuitiva. A interface mantém estado consistente durante transições entre funcionalidades. Porém, estudantes sem conhecimento prévio da sintaxe Simple precisam consultar fontes externas. A falta de exemplos pré-carregados ou *templates* de código aumenta curva de aprendizado inicial. Implementar galeria de programas exemplo diretamente na interface facilitaria exploração guiada e aprendizagem por exemplos.

D7 - Acessibilidade (2/5): Não há suporte para leitores de tela, impedindo uso por estudantes com deficiência visual. A visualização da árvore sintática e árvore de derivação depende exclusivamente de representação gráfica visual, sem alternativa textual ou auditiva. Contraste de cores não foi testado para daltonismo. Navegação por teclado é limitada, prejudicando usuários com dificuldades motoras. A interface não oferece opções de ajuste de tamanho de fonte ou alto contraste. A fim de melhorar questões de acessibilidade, seria necessário implementar descrições textuais alternativas para elementos visuais; suporte completo a navegação por teclado com indicadores visuais de foco; compatibilidade com tecnologias assistivas; e opções de personalização visual.

D9 - Conformidade com Padrões (4/5): A plataforma implementa conformidade com padrão SCORM para integração com sistemas de gerenciamento de aprendiza-

gem, facilitando distribuição e rastreamento de uso em contextos institucionais. O código HTML5 e CSS3 segue padrões web modernos, garantindo compatibilidade entre navegadores. Entretanto, ausência de metadados educacionais estruturados dificulta catalogação em repositórios educacionais e descoberta por outros educadores.

5.6. Síntese da Avaliação Pedagógica

A avaliação LORI demonstrou que o JSimples apresenta qualidade pedagógica com média de 4,3/5. Os pontos fortes concentram-se em dimensões fundamentais para software educacional: qualidade e precisão do conteúdo (D1), alinhamento direto com objetivos de aprendizagem da disciplina (D2), e design de apresentação que facilita processamento cognitivo (D5). A capacidade motivacional (D4) é particularmente notável, transformando conceitos abstratos de compiladores em experiências visuais e interativas concretas. A elevada reusabilidade (D8) confirma versatilidade da ferramenta para múltiplos contextos educacionais, desde disciplinas introdutórias até cursos avançados.

As oportunidades de melhoria identificadas (D3, D6, D7, D9) referem-se principalmente a funcionalidades de personalização e acessibilidade universal. A ausência de *feedback* adaptativo (D3) e documentação integrada (D6) representa limitação moderada que pode ser contornada através de mediação docente em contexto de sala de aula. Por outro lado, as limitações de acessibilidade (D7) constituem barreira mais significativa para inclusão de estudantes com deficiências, aspecto que deveria ser priorizado em desenvolvimentos futuros. A conformidade parcial com padrões educacionais (D9) não compromete funcionalidade atual, mas dificulta descoberta e compartilhamento da ferramenta em repositórios acadêmicos mais amplos.

Dimensão LORI	Nota	Observação
D1: Qualidade do Conteúdo	5	Implementação fiel à especificação
D2: Alinhamento com Objetivos de Aprendizagem	5	Mapeamento direto para objetivos da disciplina
D3: <i>Feedback</i> e Adaptação	4	<i>Feedback</i> claro, falta <i>linting</i>
D4: Motivação	5	Visualização interativa, experimentação ativa
D5: Design de Apresentação	5	Hierarquia visual clara, <i>syntax highlighting</i>
D6: Usabilidade da Interação	4	Falta documentação integrada
D7: Acessibilidade	2	Limitações para usuários com deficiências
D8: Reusabilidade	5	Multiplataforma, padrão SCORM
D9: Conformidade com Padrões	4	SCORM implementado, faltam metadados
Média Geral	4,3	

Tabela 4. Avaliação pedagógica do JSimples segundo LORI

6. Conclusão e trabalhos futuros

Este trabalho apresentou o desenvolvimento e avaliação do JSimples, uma plataforma web interativa para ensino de conceitos de teoria de linguagens e compiladores. A fer-

ramenta foi concebida para preencher três lacunas identificadas na revisão bibliográfica: a predominância de aplicações desktop que limitam acessibilidade, a ausência de ferramentas web com visualização interativa de árvores sintáticas combinada com execução passo a passo, e a carência de recursos didáticos em português contextualizados ao ensino brasileiro.

A implementação utilizou *stack* tecnológico moderno baseado em JavaScript e React, resultando em aplicação multiplataforma que executa diretamente no navegador sem dependências externas. A avaliação através de *frameworks* complementares (Heurísticas de Nielsen para usabilidade e LORI para qualidade pedagógica) resultou em médias de 4,0/5 e 4,3/5 respectivamente, identificando pontos fortes em aspectos como *feedback* visual, correspondência linguística e design de apresentação, além de oportunidades de melhoria em documentação, acessibilidade e *feedback* adaptativo.

As limitações identificadas referem-se à ausência de validação empírica com usuários reais, restrições de acessibilidade para estudantes com deficiências e falta de documentação integrada. Trabalhos futuros prioritários incluem condução de estudo experimental para mensurar impacto na aprendizagem, implementação de recursos de acessibilidade e desenvolvimento de funcionalidades de *linting* dentro do editor de códigos. A plataforma está disponível para uso educacional e representa base para evolução incremental através das melhorias identificadas nas avaliações conduzidas.

Referências

- Alfred, V. A., Monica, S. L., and Jeffrey, D. U. (2007). *Compilers principles, techniques tools. pearson Education.*
- Ant UED (2015). Ant design: A ui design language and react ui library. <https://ant.design/>. Acessado em: 8 de outubro de 2025.
- Bañna, K. and Benatallah, B. (2021). Is it still worth the cost to teach compiling in 2020? a pedagogical experience through hortensias compiler and virtual machine. *EasyChair*.
- Carstens, K. J., Mallon, J. M., Bataineh, M., and Al-Bataineh, A. (2021). Effects of technology on student learning. *Turkish Online Journal of Educational Technology-TOJET*, 20(1):105–113.
- Chesnevar, C. I., González, M. P., and Maguitman, A. G. (2004). Didactic strategies for promoting significant learning in formal languages and automata theory. *ACM SIGCSE Bulletin*, 36(3):7–11.
- Free Software Foundation (2025). *Bison - GNU parser generator*. Disponível em: <https://www.gnu.org/software/bison/>. Acessado em: 8 de outubro de 2025.
- Haverbeke, M. (2018). Codemirror: versatile text editor implemented in javascript for the browser. <https://codemirror.net/>. Acessado em: 7 de outubro de 2025.
- J. Vargo, J.C. Nesbit, K. B. and Archambault, A. (2003). Learning object evaluation: Computer-mediated collaboration and inter-rater reliability. *International Journal of Computers and Applications*, pages 198–205.
- Johnson, S. C. (1975). Yacc: Yet another compiler-compiler. Computing Science Technical Report 32, Bell Laboratories, Murray Hill, New Jersey.
- Kowaltowski, T. (1983). Implementação de linguagens de programação. *Guanabara Dois*.
- Kremer, B. (2017). React d3 tree. <https://bkrem.github.io/react-d3-tree/>. Acessado em: 12 de outubro de 2025.
- Maddern, G. and Stoiber, M. (2016). Styled components: Css for the component age. <https://styled-components.com/>. Acessado em: 8 de outubro de 2025.
- Mernik, M. and Zumer, V. (2003). An educational tool for teaching compiler construction. *IEEE Transactions on Education*, 46(1):61–68.
- Meta Platforms, Inc. (2025). React 19.1. <https://react.dev/>. Acessado em: 8 de outubro de 2025.
- Nielsen, J. and Molich, R. (1990). Heuristic evaluation of user interfaces. *Proceedings of the SIGCHI conference on Human factors in computing systems*, pages 249–256.
- Oliveira, H. B. (2014). "um compilador, uma linguagem de programação e uma máquina virtual simples para o ensino de lógica de programação, compiladores e arquitetura de computadores".
- Rodger, S. H. and Finley, T. W. (2006). Jflap: an interactive formal languages and automata package. *Jones Bartlett Learning*.

- Silva, L. G. D., Silva, R. L. G., Oliveira, W. R., Dias, J. C. V., and Barbosa, S. D. J. (2023). Avaliação da experiência de uso do jflap como recurso pedagógico no ensino de linguagens formais. *Simpósio Brasileiro de Informática na Educação (SBIE)*, pages 995–1006.
- Stamenković, S. and Jovanović, N. (2023). A web-based educational system for teaching compilers. *IEEE Transactions on Learning Technologies*, pages 143–156.
- Stamenković, S., Jovanović, N., and Chakraborty, P. (2020). Evaluation of simulation systems suitable for teaching compiler construction courses. *Computer Applications in Engineering Education*, 28(3):606–625.
- Tan, L. and Wang, N. (2010). Future internet: The internet of things. *IEEE*, 5(376). 3rd international conference on advanced computer theory and engineering (ICACTE).
- Zach Carter (2009). Jison Documentation. <https://gerhobbelt.github.io/jison/docs/>. Acessado em: 23 setembro de 2025.
- Álvar Arnaiz-González, Díez-Pastor, J.-F., Ramos-Pérez, I., and García-Osorio, C. (2018). Seshat—a web-based educational resource for teaching the most common algorithms of lexical analysis. *Computer applications in engineering education*, 26(6):2255–2265.